

CloudPerso

Documentation Technique

Configuration d'un cloud personnel sur Mini PC

Mini PC (exemple : Intel Core i5, 8 Go RAM) | Ubuntu Server 24.04 LTS

Jellyfin + Caddy + Docker + Tailscale

Sessions : Semaine 1 – Semaine 4 (exemple de planning)

Document anonymisé : toutes les adresses IP, noms de domaine, identifiants et informations personnelles ont été remplacés par des valeurs d'exemple à but illustratif. La configuration présentée concerne exclusivement un serveur média personnel (Jellyfin) destiné au visionnage de contenus personnels/familiaux.

Informations du serveur (exemple)

Machine	Mini PC (exemple)
Processeur	Intel Core i5 (exemple)
RAM	8 Go DDR4
Stockage	SSD 256 Go
Système	Ubuntu Server 24.04 LTS
Utilisateur	utilisateur
IP locale	192.168.1.50 (exemple)
Box internet	Box internet du fournisseur d'accès (réseau 192.168.1.x)
Domaine	moncloud.exemple.com (DNS Cloudflare)

1. Préparation et installation du système

1.1 Analyse du matériel

La machine cible est un mini PC compact. Avant de commencer l'installation, il est nécessaire de préparer une clé USB bootable avec Ubuntu Server 24.04 LTS.

- Machine : mini PC (exemple : i5, 8 Go RAM, SSD 256 Go)
- Aucune carte WiFi intégrée détectée : la connexion réseau initiale a été établie via partage de connexion USB depuis un smartphone
- Outil utilisé pour créer la clé bootable : Rufus

1.2 Installation d'Ubuntu Server 24.04 LTS

Étapes suivies lors de l'installation :

- Démarrage sur la clé USB (boot Ubuntu Server 24.04)
- Sélection de la langue : Français / France
- Configuration réseau reportée (pas de connexion disponible à ce stade)
- Pas de proxy configuré
- Partitionnement : disque entier sélectionné
- Création du profil : nom, nom de serveur, nom d'utilisateur, mot de passe

- Installation d'OpenSSH Server coché (permet la connexion SSH après démarrage)
- Redémarrage de la machine

Après redémarrage, mise à jour initiale du système :

```
sudo apt update
```

```
sudo apt upgrade
```

1.3 Connexion réseau initiale via smartphone (partage USB)

La machine ne disposant pas de carte WiFi, la connexion initiale à internet a été établie via le partage de connexion USB d'un smartphone.

Difficulté : interface réseau non activée automatiquement

L'interface réseau du smartphone n'étant pas reconnue par défaut, il a fallu l'activer manuellement et corriger le fichier de configuration Netplan (remplacement d'un point-virgule par deux-points en première ligne).

```
ip a
```

Rôle : Lister les interfaces réseau disponibles pour identifier celle du smartphone.

✓ **Résultat :** Interface USB détectée mais inactive (exemple : enxXXXXXXXXXXXX).

```
sudo ip link set enxXXXXXXXXXXXX up
```

Rôle : Activer manuellement l'interface réseau du smartphone.

✓ **Résultat :** Interface activée.

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Rôle : Corriger le fichier de configuration réseau (remplacement d'un point-virgule par deux-points en première ligne).

✓ **Résultat :** Fichier corrigé.

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
```

Rôle : Corriger les permissions du fichier Netplan (requis par systemd-networkd).

✓ **Résultat :** Permissions corrigées.

```
sudo netplan apply
```

Rôle : Appliquer la nouvelle configuration réseau.

✓ **Résultat :** IP 172.20.10.5 assignée avec succès (exemple).

```
sudo apt install network-manager -y
```

Rôle : Installer Network Manager pour faciliter la gestion réseau future.

✓ **Résultat :** Installé. nmcli confirme l'absence de carte WiFi.

1.4 Vérification du matériel

```
lspci | grep -i wireless
```

Rôle : Vérifier la présence d'une carte WiFi.

✓ **Résultat :** Aucun résultat : pas de carte WiFi intégrée. Solution : utiliser un câble Ethernet ou une clé USB WiFi.

```
lsusb
```

Rôle : Lister les périphériques USB connectés.

✓ **Résultat :** Smartphone détecté.

1.5 Installation de Cockpit

Cockpit est une interface web d'administration du serveur, accessible via navigateur sur le port 9090. Elle permet de gérer le serveur sans écran branché.

```
sudo apt install cockpit -y
```

Rôle : Installer Cockpit.

✓ Résultat : Installé.

```
sudo systemctl enable --now cockpit.socket
```

Rôle : Activer et démarrer le service Cockpit au démarrage.

✓ Résultat : Service actif. Cockpit accessible sur <http://172.20.10.5:9090> (réseau local uniquement à ce stade, exemple).

2. Installation de Docker et déploiement de Jellyfin

2.1 Installation de Docker

Difficulté : fichier de dépôt corrompu par une tentative précédente

Une tentative d'installation antérieure avait laissé un fichier `docker.list` corrompu dans les sources APT, bloquant toute nouvelle installation.

```
sudo rm /etc/apt/sources.list.d/docker.list
```

Rôle : Supprimer le fichier corrompu pour repartir proprement.

✓ Résultat : Fichier supprimé.

```
sudo apt update
```

Rôle : Mettre à jour la liste des paquets après nettoyage.

✓ Résultat : OK.

```
curl -fsSL https://get.docker.com | sudo sh
```

Rôle : Installer Docker via le script officiel.

✓ Résultat : Docker installé et démarré avec succès.

```
sudo usermod -aG docker utilisateur
```

Rôle : Ajouter l'utilisateur au groupe docker pour exécuter les commandes sans sudo.

✓ Résultat : OK.

2.2 Structure des dossiers du projet

Organisation des dossiers pour le projet :

```
mkdir -p ~/cloudperso/caddy ~/cloudperso/jellyfin/{config,cache}  
~/cloudperso/media
```

Création du Caddyfile et du fichier `docker-compose.yml` :

```
nano ~/cloudperso/caddy/Caddyfile
```

```
nano ~/cloudperso/docker-compose.yml
```

2.3 Lancement des conteneurs et accès à Jellyfin

Difficulté : Caddy monte le Caddyfile comme un dossier au lieu d'un fichier

Docker créait le montage du Caddyfile en tant que répertoire au lieu de fichier, ce qui empêchait Caddy de démarrer. La résolution a consisté à ajouter la directive `:ro` dans le volume et à supprimer le dossier créé par erreur.

```
rm -rf ~/cloudperso/caddy/Caddyfile
```

Rôle : Supprimer le dossier créé par erreur par Docker à la place du fichier.

✓ **Résultat :** Problème résolu. Le fichier a ensuite été recréé manuellement.

Difficulté : Jellyfin inaccessible (port 8096 absent du compose)

Jellyfin était démarré mais inaccessible car le port 8096 n'avait pas été exposé dans le fichier docker-compose.yml. Après ajout du port, le service est devenu accessible.

```
docker compose down
```

Rôle : Arrêter tous les conteneurs.

✓ **Résultat :** OK.

```
docker compose up -d
```

Rôle : Relancer les conteneurs en arrière-plan.

✓ **Résultat :** Jellyfin accessible sur http://192.168.1.50:8096 (exemple).

```
docker compose ps
```

Rôle : Vérifier l'état de tous les conteneurs.

✓ **Résultat :** OK. Statut Healthy confirmé.

```
docker compose logs caddy
```

Rôle : Afficher les logs de Caddy pour diagnostiquer.

✓ **Résultat :** OK.

3. Sécurisation du serveur

3.1 SSH sécurisé par clés

L'authentification par mot de passe a été désactivée au profit des clés SSH. Cela protège contre les attaques par force brute sur le port 22.

```
sudo systemctl enable ssh --now
```

Rôle : Activer SSH au démarrage.

✓ **Résultat :** OK.

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
```

Rôle : Créer et sécuriser le dossier SSH.

✓ **Résultat :** OK. Lecture réservée au propriétaire.

```
chmod 600 ~/.ssh/authorized_keys
```

Rôle : Sécuriser le fichier de clés autorisées.

✓ **Résultat :** OK.

Modifications appliquées dans /etc/ssh/sshd_config :

```
PasswordAuthentication no    # Désactive la connexion par mot de passe
```

```
PermitRootLogin no           # Désactive la connexion en root
```

```
PubkeyAuthentication yes     # Autorise uniquement les clés SSH
```

3.2 Pare-feu UFW

UFW (Uncomplicated Firewall) a été configuré pour bloquer tout trafic entrant par défaut, en n'autorisant que les ports nécessaires.

```
sudo apt install ufw -y
```

Rôle : Installer UFW.

✓ Résultat : OK.

```
sudo ufw default deny incoming
```

Rôle : Bloquer toutes les connexions entrantes par défaut.

✓ Résultat : OK.

```
sudo ufw default allow outgoing
```

Rôle : Autoriser toutes les connexions sortantes.

✓ Résultat : OK.

Ports autorisés :

```
sudo ufw allow 22      # SSH
```

```
sudo ufw allow 80      # HTTP
```

```
sudo ufw allow 443     # HTTPS
```

```
sudo ufw allow 8096    # Jellyfin
```

```
sudo ufw allow 9090    # Cockpit
```

```
sudo ufw allow 32800   # Port externe custom
```

```
sudo ufw enable
```

Rôle : Activer le pare-feu.

✓ Résultat : OK. Règles actives confirmées avec `sudo ufw status verbose`.

3.3 Fail2Ban — Protection anti force brute

Fail2Ban surveille les logs SSH et bloque automatiquement les IPs qui échouent plusieurs fois à s'authentifier.

```
sudo apt install fail2ban -y
```

Rôle : Installer Fail2Ban.

✓ Résultat : OK.

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

Rôle : Copier la config par défaut avant modification (bonne pratique).

✓ Résultat : OK.

Configuration appliquée dans la section [sshd] de `jail.local` :

```
enabled = true      # Active la protection SSH
```

```
maxretry = 5        # Bloque après 5 tentatives ratées
```

```
bantime = 3600      # Bloque l'IP pendant 1 heure
```

```
findtime = 600      # Fenêtre de comptage de 10 minutes
```

```
sudo systemctl enable fail2ban --now
```

Rôle : Activer Fail2Ban au démarrage.

✓ Résultat : OK.

3.4 Samba — Partage réseau local

Samba permet d'accéder au dossier media depuis Windows sur le réseau local, via l'interface standard de partage de fichiers Windows.

```
sudo apt install samba -y
```

Rôle : Installer Samba.

✓ Résultat : OK.

```
sudo smbpasswd -a utilisateur
```

Rôle : Créer le mot de passe Samba pour l'utilisateur.

✓ Résultat : OK.

Configuration ajoutée en bas de /etc/samba/smb.conf :

```
[Media]
```

```
path = /home/utilisateur/cloudperso/media
```

```
browseable = yes
```

```
read only = no
```

```
guest ok = no
```

```
valid users = utilisateur
```

Accessible depuis Windows via : \\192.168.1.50\Media (exemple)

4. SSL, DNS et accès depuis internet — Session (exemple)

4.1 Contexte : objectif et problèmes identifiés

L'objectif de cette session est de rendre le service accessible depuis n'importe quel appareil connecté à internet via <https://moncloud.exemple.com>. Trois problèmes principaux ont été identifiés.

Problème 1 — le fournisseur d'accès bloque les ports 80 et 443

La box internet ne laisse pas passer le trafic entrant sur les ports 80 (HTTP) et 443 (HTTPS) depuis internet. Ce sont pourtant les ports standards pour les sites web et les certificats SSL.

Solution retenue : utiliser un port alternatif (exemple : 32800), moins courant et non bloqué par le fournisseur.

Problème 2 — Let's Encrypt ne peut pas valider le domaine

Let's Encrypt délivre des certificats SSL gratuits en vérifiant que le domaine appartient bien au demandeur. La méthode par défaut (HTTP-01) nécessite que le port 80 soit accessible publiquement, ce que le fournisseur bloque.

Solution retenue : utiliser la méthode DNS-01 qui valide le domaine via les enregistrements DNS Cloudflare, sans avoir besoin du port 80. Cela nécessite de compiler une version spéciale de Caddy avec le plugin Cloudflare.

Problème 3 — les conteneurs Docker n'ont pas accès à internet

La compilation de Caddy avec le plugin Cloudflare requiert que Docker puisse télécharger des dépendances depuis internet. Or les conteneurs Docker ne parvenaient pas à sortir sur le réseau.

4.2 Diagnostic du problème réseau Docker

Une série de tests a permis de localiser précisément le blocage réseau.

```
curl -v https://acme-v02.api.letsencrypt.org/directory
```

Rôle : Vérifier si le serveur peut contacter Let's Encrypt.

✓ **Résultat** : HTTP 200 reçu. Le serveur atteint Let's Encrypt sans problème (TCP fonctionne).

```
docker run --rm alpine ping -c 2 8.8.8.8
```

Rôle : Tester si un conteneur Docker peut envoyer des paquets vers internet.

✗ **Résultat** : 100% packet loss. Les conteneurs ne peuvent pas sortir sur internet.

```
docker run --rm alpine traceroute 8.8.8.8
```

Rôle : Localiser où les paquets se bloquent.

✗ **Résultat** : Les paquets atteignent le serveur puis s'arrêtent (!H = Host Unreachable).

```
sudo ufw disable
```

Rôle : Tester si le pare-feu est responsable du blocage.

✗ **Résultat** : Pare-feu désactivé mais le problème persiste. UFW n'est pas en cause.

```
cat /proc/sys/net/ipv4/ip_forward
```

Rôle : Vérifier si le transfert de paquets entre interfaces est activé.

✓ **Résultat** : Retourne 1 = activé. Le routage est bien configuré.

```
ping -c 2 8.8.8.8
```

Rôle : Tester si le serveur lui-même peut pinguer internet.

✗ **Résultat** : Destination Host Unreachable. Le serveur ne peut pas non plus pinguer.

```
curl -I https://google.com
```

Rôle : Tester si le serveur peut accéder à internet en HTTPS (TCP).

✓ **Résultat** : HTTP/2 301 = succès. La box bloque ICMP (ping) mais pas TCP.

Conclusion du diagnostic : la box internet bloque ICMP (ping) mais pas TCP/HTTPS. Les conteneurs Docker, eux, ne peuvent pas sortir à cause d'un problème de résolution DNS, pas de routage.

4.3 Correction à tenter (en attente)

- Réactiver UFW : `sudo ufw enable`
- Relancer le build Caddy avec plugin Cloudflare : `docker compose build caddy`
- Si le build réussit : `docker compose up -d` puis tester `https://moncloud.exemple.com:32800`
- Si le build échoue encore : pré-builder l'image Caddy depuis le serveur hôte (hors Docker)

5. Résolution SSL et Tailscale — Session (exemple)

5.1 Accès à distance via Cockpit

La session commence par une connexion SSH depuis un PC distant, puis vérification que Cockpit est accessible pour éviter de brancher un écran.

```
ssh utilisateur@192.168.1.50
```

Rôle : Se connecter au serveur à distance.

✓ **Résultat** : Connexion réussie. Ubuntu 24.04 confirmé, IP locale validée (exemple).

```
dpkg -l cockpit
```

Rôle : Vérifier si Cockpit est installé.

✓ **Résultat** : Cockpit installé et fonctionnel.

```
systemctl status cockpit.socket
```

Rôle : Vérifier si le service Cockpit est actif.

✓ **Résultat :** Active (listening) sur le port 9090.

```
sudo ufw allow 9090/tcp && sudo ufw reload
```

Rôle : Ouvrir le port 9090 dans UFW pour accéder à Cockpit depuis le navigateur.

✓ **Résultat :** Port 9090 ouvert. Cockpit accessible sur https://192.168.1.50:9090 (exemple).

5.2 Diagnostic SSL de Caddy (suite)

```
docker run --rm --network host alpine ping -c 2 8.8.8.8
```

Rôle : Tester le ping en utilisant directement le réseau du serveur (sans bridge Docker).

✗ **Résultat :** 100% packet loss. Confirme que le problème ne vient pas de Docker mais du serveur lui-même.

Difficulté : les conteneurs Docker ne résolvent pas les noms de domaine

La box internet bloque l'UDP sortant vers les DNS externes (8.8.8.8, 1.1.1.1). Les conteneurs Docker ne peuvent donc pas résoudre les noms de domaine pour télécharger les dépendances.

```
nslookup acme-v02.api.letsencrypt.org 8.8.8.8
```

Rôle : Tester si le DNS externe est accessible en UDP depuis le serveur.

✗ **Résultat :** Timeout. La box bloque l'UDP sortant vers les DNS externes.

```
nslookup acme-v02.api.letsencrypt.org 127.0.0.53
```

Rôle : Vérifier que le résolveur DNS local Ubuntu (systemd-resolved) fonctionne.

✓ **Résultat :** Réponse reçue via DoH (DNS over HTTPS). Le DNS local fonctionne.

Solution : forcer Docker à utiliser le DNS local Ubuntu

La solution consiste à configurer Docker pour qu'il utilise le résolveur DNS local d'Ubuntu (127.0.0.53) au lieu des DNS externes bloqués.

Contenu de /etc/docker/daemon.json :

```
{"dns": ["127.0.0.53"], "ipv6": false}
```

```
sudo nano /etc/docker/daemon.json
```

Rôle : Créer le fichier de configuration Docker avec le DNS local.

✓ **Résultat :** Fichier créé.

```
sudo systemctl restart docker
```

Rôle : Redémarrer Docker pour appliquer la nouvelle configuration DNS.

✓ **Résultat :** Docker redémarré. Les conteneurs utilisent maintenant 127.0.0.53.

5.3 Compilation de Caddy avec le plugin Cloudflare

Méthode 1 (échec) : docker compose build caddy

```
docker compose build caddy
```

Rôle : Compiler l'image Docker Caddy avec le plugin Cloudflare.

✗ **Résultat :** Échec. Même avec le DNS corrigé, les conteneurs ne peuvent pas télécharger les dépendances au moment du build.

Méthode 2 (succès) : compilation directe sur le serveur

La compilation a été réalisée directement sur le serveur (hors Docker), ce qui contourne le problème de réseau des conteneurs lors du build.

```
sudo apt install golang-go -y
```

Rôle : Installer Go (langage de programmation) sur le serveur.

✓ **Résultat :** Go installé avec succès.

```
go install github.com/caddyserver/xcaddy/cmd@latest
```

Rôle : Installer xcaddy, l'outil officiel pour compiler Caddy avec des plugins personnalisés.

✓ **Résultat :** xcaddy installé dans ~/go/bin/.

```
~/go/bin/xcaddy build --with github.com/caddy-dns/cloudflare --output  
~/cloudperso/caddy/caddy
```

Rôle : Compiler Caddy avec le plugin Cloudflare. Le binaire est placé dans ~/cloudperso/caddy/.

✓ **Résultat :** Build réussi. Caddy compilé avec plugin Cloudflare.

5.4 Obtention du certificat SSL

Après correction du DNS Docker et redéploiement des conteneurs, Caddy a pu contacter Let's Encrypt et obtenir le certificat SSL via la méthode DNS-01.

```
docker compose down && docker compose up -d
```

Rôle : Redémarrer tous les conteneurs pour appliquer la nouvelle config DNS Docker.

✓ **Résultat :** OK. Caddy obtient son certificat SSL avec succès.

```
docker logs caddy 2>&1 | grep -E 'certificate|error|warn|obtain'
```

Rôle : Filtrer les logs Caddy pour vérifier l'obtention du certificat.

✓ **Résultat :** certificate obtained successfully pour moncloud.exemple.com (exemple).

5.5 Problème d'accès externe : CG-NAT

Même avec le certificat SSL obtenu et un port alternatif ouvert, le domaine reste inaccessible depuis internet. Un nouveau problème est identifié : le fournisseur d'accès utilise le CG-NAT (Carrier Grade NAT), ce qui signifie que la box ne dispose pas d'une IPv4 publique dédiée.

```
curl -4 ifconfig.me
```

Rôle : Récupérer l'IPv4 publique du serveur.

✗ **Résultat :** Échec. Le serveur n'a pas d'IPv4 publique : le fournisseur utilise le CG-NAT.

```
curl ifconfig.me
```

Rôle : Récupérer l'IP publique (toutes versions).

✗ **Résultat :** Retourne une IPv6. Confirme l'absence d'IPv4 publique dédiée.

Le CG-NAT signifie que plusieurs clients du fournisseur partagent la même IPv4 publique. Le port forwarding dans la box ne fonctionne pas car la box n'est pas directement exposée sur internet.

5.6 Solution de contournement : Tailscale

Tailscale est un VPN mesh qui crée un tunnel sécurisé entre les appareils sans avoir besoin d'IPv4 publique ni de port forwarding. Il contourne complètement le CG-NAT.

```
tailscale status
```

Rôle : Vérifier si Tailscale est installé et connecté.

✓ **Résultat :** Serveur connecté avec IP Tailscale 100.100.100.1 (exemple).

```
sudo tailscale up
```

Rôle : Activer et connecter le serveur au réseau Tailscale.

✓ **Résultat :** Serveur connecté.

```
curl -v http://localhost:8096/health
```

Rôle : Vérifier que Jellyfin répond correctement en local.

✓ **Résultat :** HTTP 200 OK. Jellyfin fonctionne.

Jellyfin est désormais accessible via l'IP Tailscale du serveur depuis tous les appareils Tailscale connectés au même compte.

6. État final du projet et prochaines étapes

6.1 Tableau de bord des services (exemple)

Service	Statut	Accès (exemple)
Cockpit (gestion serveur)	Fonctionnel	https://192.168.1.50:9090
SSH	Fonctionnel	ssh utilisateur@192.168.1.50
Jellyfin (local)	Fonctionnel	http://192.168.1.50:8096
Jellyfin (Tailscale)	Fonctionnel	http://100.100.100.1:8096
Certificat SSL	Obtenu	acme-v02.api.letsencrypt.org
Samba (partage Windows)	Fonctionnel	\\192.168.1.50\Media
Domaine public (internet)	Bloqué (CG-NAT)	Non accessible

6.2 Prochaines étapes

- Configurer le Caddyfile avec reverse_proxy localhost:8096 pour servir Jellyfin via Caddy
- Tester l'accès complet via Tailscale sur tous les appareils (smartphone, PC, TV)
- Configurer une IP statique (connexion Ethernet directe à la box + port forwarding)
- Vérifier l'espace disque visible et étendre la partition si nécessaire
- Connecter un disque dur externe pour étendre le stockage média
- Upgrade RAM
- Déployer les prochains services : Nextcloud, Immich, gestion de documents, gestionnaire de mots de passe, blocage de publicités réseau

6.3 Ressources utilisées

Documentation officielle Ubuntu Server 24.04 LTS

Script d'installation Docker officiel : <https://get.docker.com>

xcaddy (compilation Caddy custom) : <https://github.com/caddyserver/xcaddy>

Plugin DNS Cloudflare pour Caddy : <https://github.com/caddy-dns/cloudflare>

Tailscale : <https://tailscale.com>